

PROMÉTHÉE — Guide développeur

Table of Contents

Ajouter une méthode	1
Structure d'un module méthode	1
Implémenter le trait <code>RuleTrait</code>	1
Enregistrer la méthode	2
Écriture immédiate ou groupée ?	2

Ajouter une méthode

Ce guide décrit la procédure à suivre pour ajouter une nouvelle méthode de sécurisation à PROMÉTHÉE. La méthode `windows_optional_feature` (`src/methods/windows_optional_feature/`) constitue l'exemple de référence le plus simple.

Structure d'un module méthode

Chaque méthode est un module Rust dans `src/methods/<nom>/`, composé de quatre fichiers :

Fichier	Rôle
<code>mod.rs</code>	Déclaration du module et ré-export des types publics.
<code>data_structures.rs</code>	Définition de la structure porteuse et des énumérations utilisées par la méthode (paramètres, énumérations d'action).
<code>method.rs</code>	Implémentation du trait <code>RuleTrait</code> (<code>execute</code> , <code>current_value</code> , <code>to_raw</code> , <code>from_raw</code>).
<code>utils.rs</code>	Fonctions auxiliaires (appels PowerShell, parsing, sérialisation).

Implémenter le trait `RuleTrait`

Le trait `RuleTrait` est défini dans `src/traits/rule.rs`. Votre structure doit implémenter les quatre méthodes suivantes :

Méthode	Rôle
<code>fn execute(&self) → R<()></code>	Applique la règle au système. Appelée par la sous-commande <code>apply</code> .
<code>fn current_value(&self) → R<RawMethod></code>	Relève l'état actuel du système pour les champs de la règle et le retourne sous forme de <code>RawMethod</code> . Appelée par <code>backup</code> .
<code>fn to_raw(&self, compare_mode: bool) → RawMethod</code>	Sérialise l'état interne de la règle en <code>RawMethod</code> . Le paramètre <code>compare_mode</code> active une normalisation (minuscules, tri) destinée à la comparaison d'audit.
<code>fn from_raw(raw: RawMethod) → R<Self></code>	Construit une instance à partir d'un <code>RawMethod</code> (les six colonnes <code>method</code> , <code>target</code> , <code>option1</code> , <code>option2</code> , <code>scope</code> , <code>action</code> issues du CSV).

La structure `RawMethod` est définie dans `src/methods/method/data_structures.rs`.

Enregistrer la méthode

Pour que PROMÉTHÉE reconnaisse votre méthode, ajoutez une entrée dans le registre `FROM_RAW_REGISTRY` du fichier `src/methods/mod.rs`. Utilisez la macro `method_fromraw_map!` :

```
method_fromraw_map!(  
    "nom_methode" => MaStructure::from_raw  
);
```

La clé doit correspondre exactement à la valeur attendue dans la colonne `method` du fichier CSV.

Écriture immédiate ou groupée ?

Deux stratégies d'écriture coexistent dans PROMÉTHÉE :

- **Écriture immédiate** : chaque règle `execute()` appelle directement PowerShell ou une API Windows. Utilisé par les méthodes `appx_package`, `firewall`, `local_group`, `local_user`, `windows_capability` et `windows_optional_feature`. C'est l'approche la plus simple, adaptée aux opérations rapides et indépendantes.
- **Écriture groupée** : les règles accumulent leurs modifications dans un cache global (`OnceLock<Mutex<Vec<...>>>`) pendant la phase `execute()`, puis une fonction `update_*` est appelée en fin d'exécution pour écrire toutes les modifications en une seule opération. Utilisé par `lgpo`, `secedit`, `advanced_auditing`, `safer` et `service`. Cette approche est nécessaire lorsque l'outil sous-jacent (`LGPO.exe`) ne permet pas d'écritures incrémentales.

Si votre méthode utilise `LGPO.exe`, privilégiez l'écriture groupée. Sinon, l'écriture immédiate est recommandée pour sa simplicité.